

Relatório Técnico / Repositório Oficial

Compilador - Mestre Komatsu

UNIC: Universidade de Cuiabá

Matéria: Compiladores

Professor: Edson Komatsu

Integrantes: Emmanuel Duarte de Oliveira, Sandro Delmondes dos Anjos, Leandro Augusto Mestre Santana

Local/Ano: Cuiabá/MT - 2026

1. Introdução

Este projeto implementa um compilador didático completo, desenvolvido para a disciplina de Compiladores, cobrindo todas as principais fases do processo de tradução de uma linguagem de programação:

- Análise léxica: identificação de tokens a partir do código-fonte.
- Análise sintática: construção da Árvore Sintática Abstrata (AST) com base em uma gramática formal em EBNF (ISO/IEC 14977).
- Análise semântica: verificação de regras de uso de identificadores (ex.: variáveis declaradas/atribuídas antes do uso), utilizando Tabela de Símbolos.
- Geração de código alvo (Backend): transpilação da AST validada para um arquivo em linguagem C (saida.c).
- Integração com GCC: compilação do código C gerado em um executável nativo.

A linguagem suportada é propositalmente simples e focada em fins educacionais, contendo:

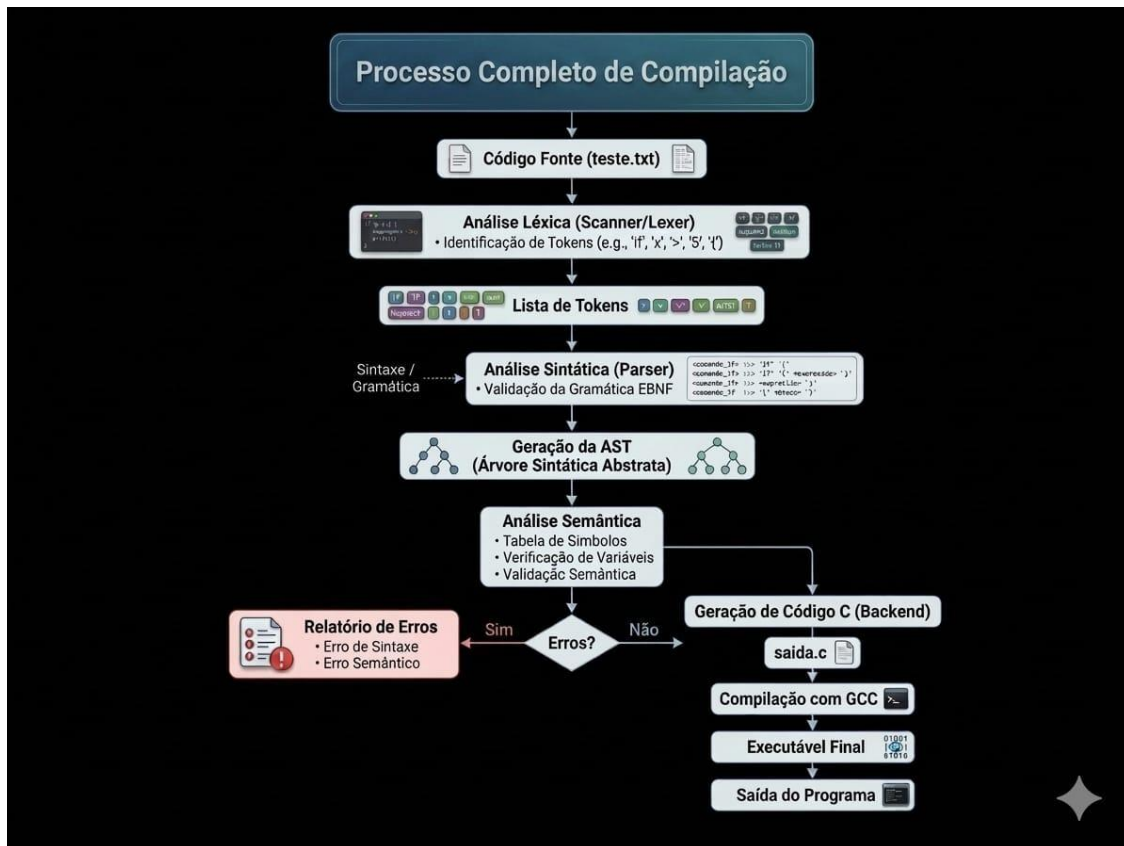
- Atribuição de variáveis, com declaração implícita.
- Saída de dados via comando print.
- Estruturas de controle de fluxo: if/else e laços while.
- Operações aritméticas e relacionais básicas, trabalhando com inteiros.

O compilador foi implementado em Go, utilizando a biblioteca Participle para o analisador sintático descendente (Top-Down), com a gramática formalizada em EBNF diretamente mapeada para estruturas de dados (structs) da linguagem Go.

2. Arquitetura do Compilador

O compilador é implementado de forma modular, concentrado em um arquivo principal (main.go), mas claramente separado em fases lógicas:

Fluxo de Fases do Compilador:



2.1. Módulo Léxico (Scanner)

- Implementado com expressões regulares integradas à biblioteca Participle via `lexer.SimpleRule`.
- Responsável por converter a sequência de caracteres do arquivo teste.txt em uma sequência de tokens (Keyword, Ident, Number, Operator, Punct, Whitespace).
- Erros léxicos: caracteres que não se enquadram em nenhum token válido são reportados explicitamente (exemplo em erro_lexico.txt).

2.2. Módulo Sintático (Parser)

- Implementado com Participle em Go.
- A gramática EBNF é mapeada diretamente para structs Go, permitindo a construção de uma Árvore Sintática Abstrata (AST).
- Utiliza um analisador descendente (Top-Down).
- Erros sintáticos (por exemplo, parênteses ou chaves não fechados) são detectados nesta fase (exemplo em erro_sintatico.txt).

2.3. Módulo Semântico

- Percorre recursivamente a AST construindo e consultando uma Tabela de Símbolos.
- Regras principais:
 - Variáveis são criadas implicitamente na primeira atribuição.
 - É proibido usar uma variável em expressões ou em print antes de ter sido atribuída.
- Erros semânticos são gerados quando uma variável “fantasma” é utilizada (exemplo em erro_semantico.txt).

2.4. Módulo de Backend (Geração de Código C)

- Percorre a AST validada e gera um arquivo C padrão (saida.c).
- Mapeia diretamente:
 - Atribuições para atribuições em C.
 - print(ident) para uma chamada de printf apropriada em C (por exemplo, printf("%d\n", ident);).
 - Estruturas if/else e while para seus equivalentes em C.
- O compilador Go atua como um transpilador, delegando ao GCC a geração do código de máquina final a partir de saida.c.

3. Especificação Léxica (Tokens)

A análise léxica é regida por um conjunto de expressões regulares que identificam os símbolos básicos da linguagem:

- Keyword:
 - Regex: `\b(if|else|while|print)\b`
 - Descrição: Palavras reservadas de controle de fluxo e comando de saída.
- Ident:
 - Regex: `[a-zA-Z_][a-zA-Z0-9_]*`
 - Descrição: Identificadores de variáveis (devem começar com letra ou `_`).
- Number:
 - Regex: `\d+`
 - Descrição: Literais numéricos inteiros (sequência de dígitos 0–9).
- Operator:
 - Regex: `[=+*/><!-]`
 - Descrição: Operadores de atribuição, aritméticos e relacionais básicos.
- Punct:
 - Regex: `[{}()]`
 - Descrição: Delimitadores utilizados para blocos e agrupamento de expressões.

- Whitespace:
 - Regex: \s+
 - Descrição: Espaços, tabs e quebras de linha.
 - Tratamento: Reconhecidos pelo léxico e ignorados pelo analisador sintático.

4. Gramática Formal da Linguagem (EBNF - ISO/IEC 14977)

A seguir, a gramática formal da linguagem, escrita em EBNF, que também orienta a construção da AST no parser:

```
ebnf

(* Estrutura Principal *)
Programa  = { Instrucao } ;
Instrucao = Atribuicao
          | Print
          | If
          | While
          ;

(* Regras de Produção de Instruções Básicas *)
Atribuicao = Ident , "=" , Expressao ;
Print     = "print" , "(" , Ident , ")" ;
If        = "if" , "(" , Condicao , ")" , "{"
          , { Instrucao }
          , "}"
          , [ "else" , "{"
          , { Instrucao }
          , "}"
          ]
          ;
While     = "while" , "(" , Condicao , ")" , "{"
          , { Instrucao }
          , "}"
          ;

(* Expressões e Termos *)
Condicao   = Termo , Operator , Termo ;
Expressao = Termo , [ Operator , Termo ] ;
Termo     = Number
          | Ident
          ;

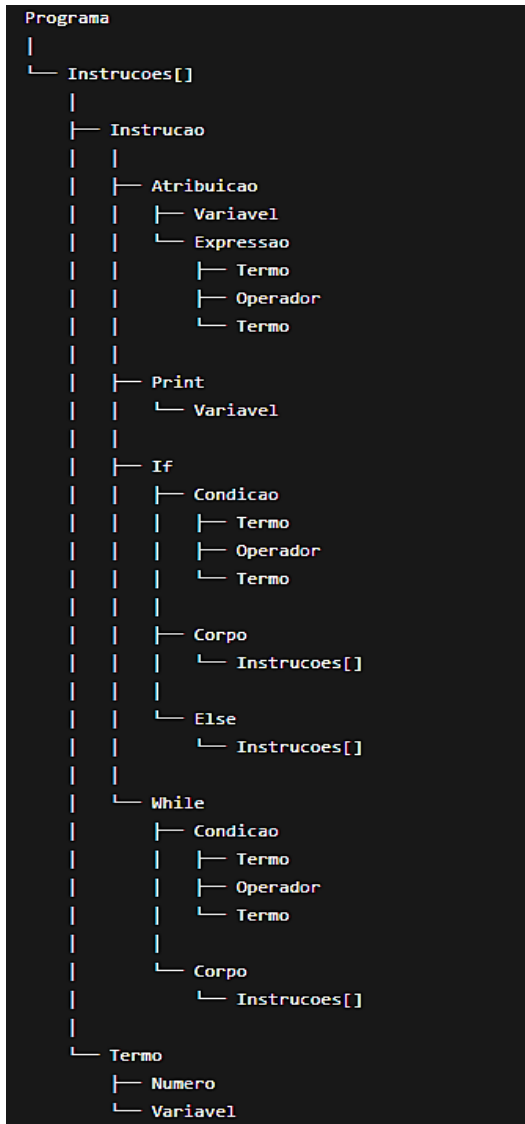
(* Unidades Léxicas - Correspondência Conceitual *)
Ident     = /* identificador: [a-zA-Z_][a-zA-Z0-9_]* */ ;
Number    = /* número inteiro: [0-9]+ */ ;
Operator  = "=" | "+" | "-" | "*" | "/" | ">" | "<" | "!" ;
```

Observações sobre a gramática:

- Programa é a raiz: é uma sequência de 0 ou mais instruções.
- Instrução pode ser:
 - Atribuição
 - Print
 - If
 - While

- Expressão é propositalmente simples: um Termo opcionalmente seguido de Operator e outro Termo.
Ex.: $x + 1$, 10 , $y * z$.
- Condição compara dois Termo com um Operator relacional/igualdade.
Ex.: $x > y$, $x = 10$.

4.1 - Estrutura da AST do Compilador



4.2 Estrutura da Árvore Sintática Abstrata (AST)

A AST (Abstract Syntax Tree) representa a estrutura hierárquica produzida pelo analisador sintático a partir da gramática EBNF implementada com a biblioteca Participle. A raiz da árvore é o nó programa, que contém uma sequência de instruções. Cada instrução pode representar uma atribuição, um comando de impressão, uma estrutura condicional (if/else) ou uma estrutura de repetição (while). Os nós expressão, condição e termo são utilizados para representar operações aritméticas, relacionais e operandos da linguagem. Essa estrutura é posteriormente utilizada pela análise semântica e pela geração do código C.

5. Regras de Sintaxe e Semântica

5.1. Regras Sintáticas

- Cada instrução deve:
 - Estar em uma linha separada, ou
 - Estar claramente delimitada por chaves em blocos (`{ ... }`).
- Espaços em branco (espaços, tabs, quebras de linha) são ignorados do ponto de vista sintático.
- Blocos de código (em `if`, `else` e `while`) são sempre delimitados por `{` e `}`.
- Parênteses em condições e chamadas de `print` são obrigatórios:
 - `if (condição) { ... }`
 - `while (condição) { ... }`
 - `print(ident)`

5.2. Regras Semânticas

- Tipos de dados suportados: apenas inteiros (Number).
- Declaração de variáveis: é implícita na primeira atribuição.
Ex.: `x = 10` cria (instancia) a variável `x` e atribui o valor 10.
- Uso de variáveis:
 - É estritamente proibido utilizar uma variável:
 - Em expressões aritméticas,
 - Em condições lógicas/relacionais,
 - Em comandos `print`,
 - antes de ela ter recebido uma atribuição prévia.
 - O compilador realiza análise semântica com uso de Tabela de Símbolos:
 - Se uma variável `z` aparece em `soma = x + z` sem ter recebido valor antes, é gerado erro semântico.
- Escopo de variáveis:
 - Para fins didáticos, assume-se escopo global simples; todas as variáveis compartilham o mesmo escopo em todo o programa.

6. Exemplos de Código

6.1. Exemplo Válido Completo (codigo_funcionando_exemplo.txt)

```
(base) artíficer@pop-os:~/Documentos/GitHub/Projeto_Compilador_Faculdades$ ./run.sh
Tipo: Keyword Valor: "while" (linha 8, coluna 1)
Tipo: Punct Valor: "(" (linha 8, coluna 7)
Tipo: Ident Valor: "x" (linha 8, coluna 8)
Tipo: Operator Valor: "<" (linha 8, coluna 10)
Tipo: Number Valor: "100" (linha 8, coluna 12)
Tipo: Punct Valor: ")" (linha 8, coluna 15)
Tipo: Punct Valor: "{" (linha 8, coluna 17)
Tipo: Ident Valor: "x" (linha 9, coluna 5)
Tipo: Operator Valor: "=" (linha 9, coluna 7)
Tipo: Ident Valor: "x" (linha 9, coluna 9)
Tipo: Operator Valor: "+" (linha 9, coluna 11)
Tipo: Number Valor: "1" (linha 9, coluna 13)
Tipo: Punct Valor: "}" (linha 10, coluna 1)
✅ Sucesso! O compilador encontrou 4 instrução(ões) raiz no código.

🔧 Iniciando Análise Semântica...
✅ Análise Semântica aprovada! Nenhuma variável usada sem declaração.

🌟 Gerando código executável (C)...
✅ Código gerado com sucesso no arquivo 'saida.c'!

=== SAÍDA DO PROGRAMA ===
20
(base) artíficer@pop-os:~/Documentos/GitHub/Projeto_Compilador_Faculdades$
```

Comportamento esperado:

1. São geradas as variáveis x e y.
2. A condição if (x > y) é avaliada:
 - Como 10 > 20 é falso, o bloco else é executado, imprimindo o valor de y.
3. O laço while (x < 100) incrementa x até que x atinja 100.

6.2. Exemplo de Erro Léxico (erro_lexico.txt)

```
(base) artíficer@pop-os:~/Documentos/GitHub/Projeto_Compilador_Faculdades$ ./run.sh
=== INICIANDO COMPILADOR ===
🔍 O compilador leu o seguinte texto do arquivo:
[
x = 10
y = 20 @ 5]

🔍 Tokens encontrados:
Tipo: Ident Valor: "x" (linha 2, coluna 1)
Tipo: Operator Valor: "=" (linha 2, coluna 3)
Tipo: Number Valor: "10" (linha 2, coluna 5)
Tipo: Ident Valor: "y" (linha 3, coluna 1)
Tipo: Operator Valor: "=" (linha 3, coluna 3)
Tipo: Number Valor: "20" (linha 3, coluna 5)
❌ Erro Léxico: teste.txt:3:8: lexer: invalid input text "@ 5"

=== SAÍDA DO PROGRAMA ===
20
(base) artíficer@pop-os:~/Documentos/GitHub/Projeto_Compilador_Faculdades$ ./run.sh
=== INICIANDO COMPILADOR ===
🔍 O compilador leu o seguinte texto do arquivo:
[x = 10
soma = x + z
print(soma)]
```

- O caractere @ não pertence a nenhum token válido definido na especificação léxica.

- O analisador léxico deve emitir um erro léxico, indicando a posição aproximada do caractere inválido.

6.3. Exemplo de Erro Sintático (erro_sintatico.txt)

```
(base) artifer@pop-os:~/Documentos/GitHub/Projeto_Compilador_Faculdade$ ./run.sh
if (x > 5 {
    print(x)
})

Tokens encontrados:
Tipo: Ident    Valor: "x"      (linha 1, coluna 1)
Tipo: Operator Valor: "="    (linha 1, coluna 3)
Tipo: Number   Valor: "10"     (linha 1, coluna 5)
Tipo: Keyword  Valor: "if"     (linha 2, coluna 1)
Tipo: Punct    Valor: "{"      (linha 2, coluna 4)
Tipo: Ident    Valor: "x"      (linha 2, coluna 5)
Tipo: Operator Valor: ">"      (linha 2, coluna 7)
Tipo: Number   Valor: "5"      (linha 2, coluna 9)
Tipo: Punct    Valor: "{"      (linha 2, coluna 11)
Tipo: Keyword  Valor: "print"   (linha 3, coluna 5)
Tipo: Punct    Valor: "("      (linha 3, coluna 10)
Tipo: Ident    Valor: "x"      (linha 3, coluna 11)
Tipo: Punct    Valor: ")"      (linha 3, coluna 12)
Tipo: Punct    Valor: "}"      (linha 4, coluna 1)
✗ Erro de Sintaxe na linha 2, coluna 11: teste.txt:2:11: unexpected token "{" (expected ")" "{" Instrucao* ")" ("else" "{" Instrucao* ")")
?)

=== SAÍDA DO PROGRAMA ===
20
```

- Falta um) após x > 5.
- O analisador sintático detecta essa inconsistência na estrutura da gramática e gera um erro sintático, indicando a região problemática.

6.4. Exemplo de Erro Semântico (erro_semantico.txt)

```
(base) artifer@pop-os:~/Documentos/GitHub/Projeto_Compilador_Faculdade$ ./run.sh
[x = 10
soma = x + z
print(soma)]

Tokens encontrados:
Tipo: Ident    Valor: "x"      (linha 1, coluna 1)
Tipo: Operator Valor: "="    (linha 1, coluna 3)
Tipo: Number   Valor: "10"     (linha 1, coluna 5)
Tipo: Ident    Valor: "soma"   (linha 2, coluna 1)
Tipo: Operator Valor: "="      (linha 2, coluna 6)
Tipo: Ident    Valor: "x"      (linha 2, coluna 8)
Tipo: Operator Valor: "+"      (linha 2, coluna 10)
Tipo: Ident    Valor: "z"      (linha 2, coluna 12)
Tipo: Keyword  Valor: "print"  (linha 3, coluna 1)
Tipo: Punct    Valor: "("      (linha 3, coluna 6)
Tipo: Ident    Valor: "soma"   (linha 3, coluna 7)
Tipo: Punct    Valor: ")"      (linha 3, coluna 11)
✓ Sucesso! O compilador encontrou 3 instrução(ões) raiz no código.

Iniciando Análise Semântica...
✗ Erro Semântico: variável 'z' não declarada

=== SAÍDA DO PROGRAMA ===
20
```

- A variável z nunca recebeu atribuição antes de ser usada.
- Durante a análise semântica, a Tabela de Símbolos indica ausência de declaração/atribuição de z.
- O compilador reporta um erro semântico indicando uso de variável não inicializada.

7. Configuração do Ambiente

7.1. Requisitos

- Linguagens/Compiladores:
 - Go (golang)
 - GCC (para compilar o código C gerado)
- Sistema Operacional:
 - Linux, macOS ou Windows (nativo com MinGW-w64 ou via WSL)

7.2. Instalação - Linux (Pop!_OS / Ubuntu)

No terminal:

```
bash

sudo apt update
sudo apt install golang build-essential
```

7.3. Instalação - Windows

1. Instalar Go
 - Baixar a versão mais recente em: go.dev
2. Instalar GCC
 - Opção A: Instalar MinGW-w64 e adicionar o bin do MinGW ao PATH do Windows.
 - Opção B (Recomendada): Utilizar WSL (Windows Subsystem for Linux) e executar o projeto com os mesmos comandos de Linux.

7.4. Dependências do Projeto (Go Modules)

Na pasta raiz do projeto, execute:

```
bash

go mod tidy
```

Esse comando fará o download e sincronização da biblioteca Participle utilizada na construção do parser.

8. Execução do Compilador

O compilador lê o código-fonte exclusivamente do arquivo teste.txt localizado na raiz do projeto.

8.1. Preparação do Arquivo de Entrada

Abra o arquivo teste.txt e:

1. Apague o texto explicativo existente.
2. Copie um dos exemplos da pasta /testes:

- codigo_funcionando_exemplo.txt
 - erro_lexico.txt
 - erro_sintatico.txt
 - erro_semantico.txt
3. Cole o código dentro de teste.txt.
 4. Salve o arquivo.

8.2. Método 1 - Script Automático (Linux/macOS/WSL)

No terminal, na raiz do projeto:

```
bash

./run.sh
```

Fluxo automatizado do script:

1. Executa o compilador Go (go run main.go), que:
 - Realiza análise léxica, sintática e semântica.
 - Gera o arquivo saida.c se não houver erros.
2. Chama o GCC para compilar saida.c em um executável.
3. Executa o programa final gerado.

8.3. Método 2 - VS Code com Extensão Code Runner

1. Abra teste.txt, cole o código de teste e salve.
2. Abra main.go.
3. Clique em Run Code (botão “Play” no canto superior direito do editor).
4. Verifique:
 - Saída do compilador no terminal integrado.
 - Geração do arquivo saida.c.
 - Compilação com GCC (pode ser feita manualmente conforme abaixo, se o script não estiver integrado).

8.4. Método 3 - Execução Manual (Terminal, CMD ou PowerShell)

Na raiz do projeto:

```
bash

# 1. Executa o compilador Go (gera saida.c)
go run main.go
# 2. Compila o arquivo C gerado para código de máquina
gcc saida.c -o meu_programa
# 3. Executa o programa final
./meu_programa      # Linux/macOS/WSL
meu_programa.exe    # Windows
```

9. Estrutura de Pastas e Arquivos Importantes

- `main.go`
Arquivo principal do compilador, contendo:
 - Definição das structs que representam a AST.
 - Configuração do lexer (regras léxicas).
 - Configuração do parser (gramática EBNF via Participle).
 - Funções de análise semântica (Tabela de Símbolos).
 - Funções de geração de código C (`saida.c`).
- `teste.txt`
Arquivo de entrada do compilador. Deve conter apenas o código fonte em linguagem didática definida neste projeto.
- `saida.c`
Arquivo C gerado pelo backend do compilador. É a representação transpilada da AST validada.
- `/testes`
Pasta com casos de teste:
 - `codigo_funcionando_exemplo.txt` - caso de uso válido e completo.
 - `erro_lexico.txt` - demonstração de erro léxico.
 - `erro_sintatico.txt` - demonstração de erro sintático.
 - `erro_semantico.txt` - demonstração de erro semântico (variável não declarada).
- `run.sh` (opcional, em ambientes Unix-like)
Script auxiliar para automatizar:
 - Execução do compilador Go.
 - Compilação de `saida.c` com GCC.
 - Execução do programa final.

10. Considerações Finais

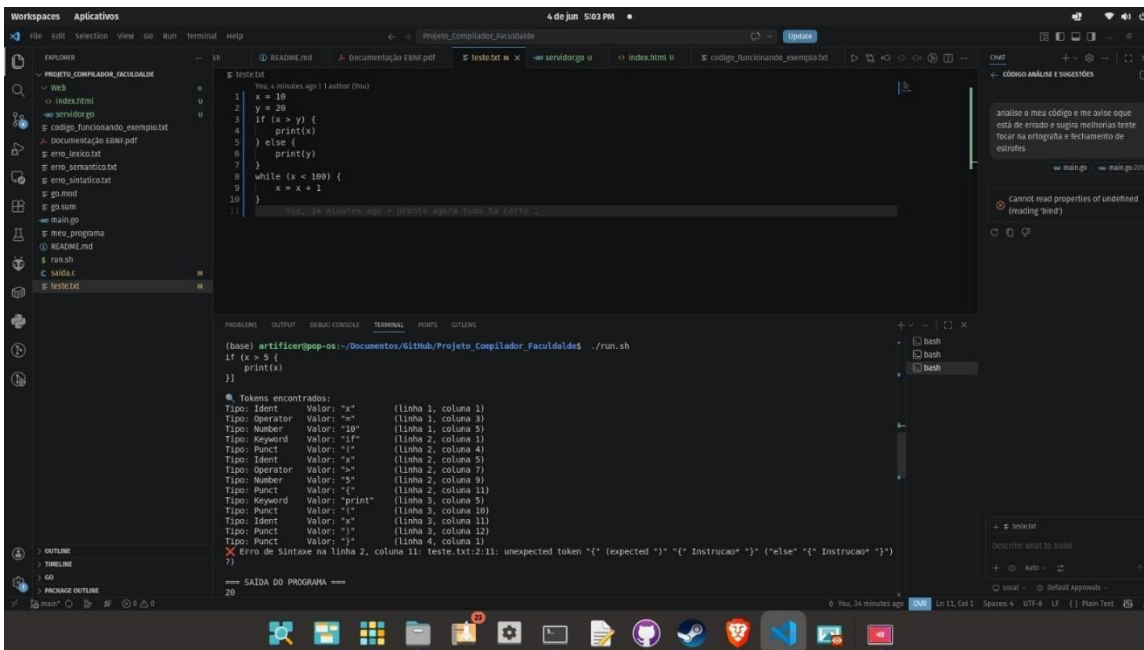
Este compilador demonstra, de forma clara e didática, o ciclo completo de construção de um sistema de tradução, desde a especificação formal da gramática (EBNF) até a geração de código executável:

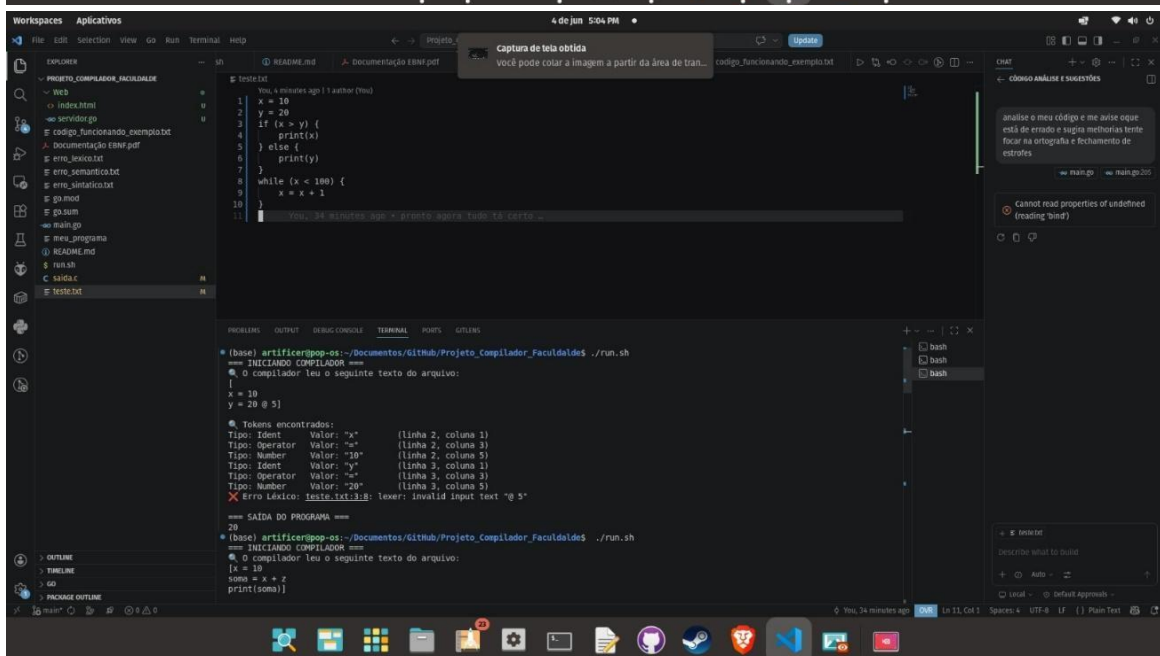
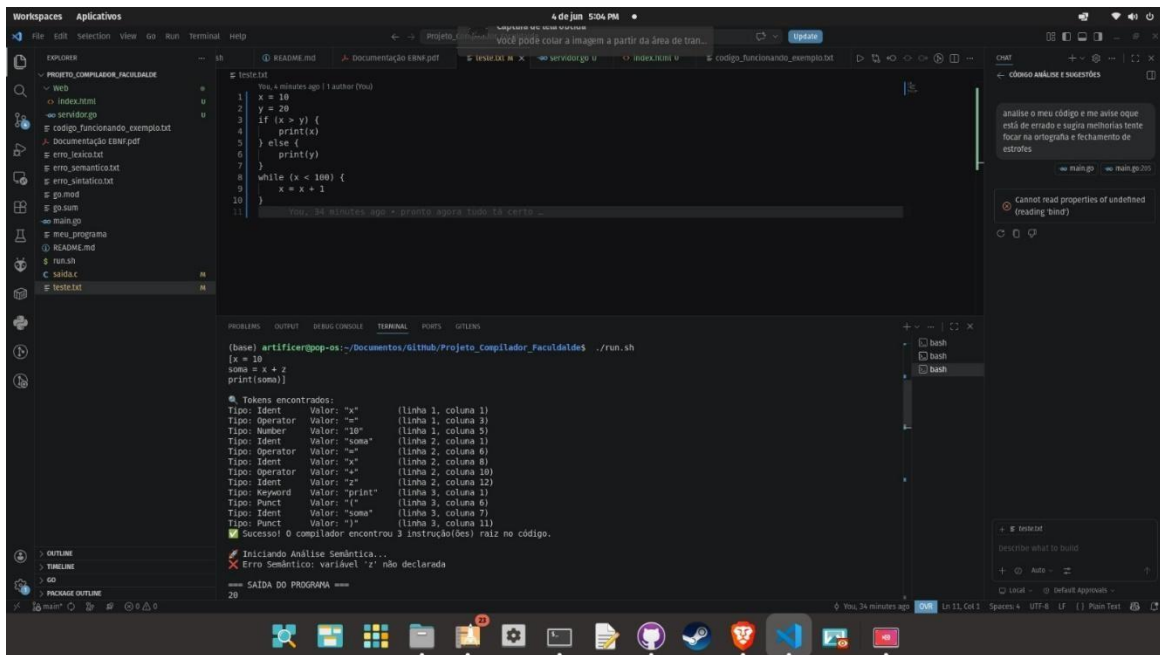
- Formalização da linguagem em EBNF (ISO/IEC 14977).
- Implementação do léxico com expressões regulares.
- Parser Top-Down com mapeamento direto da EBNF para AST via Participle.
- Verificações semânticas com Tabela de Símbolos.
- Backend que gera código C legível e facilmente extensível.
- Integração com GCC para geração do executável final.

O projeto serve como base para estudos de:

- Construção de compiladores modernos em Go.

- ## 11. Exemplos do Funcionamento do Compilador





12. Referência Bibliográfica

MEDEIROS, João Antonio. *Compiladores para Humanos*. Disponível em: johnidm.gitbooks.io Acesso em: jun. 2026.